

Fonctions en assembleur

Sommaire

Nettoyer la pile

[Conventions d'appel AMD 64](#)

[Aligner la pile](#)

[Instructions logiques bit à bit](#)

[Mélanger nasm et C](#)

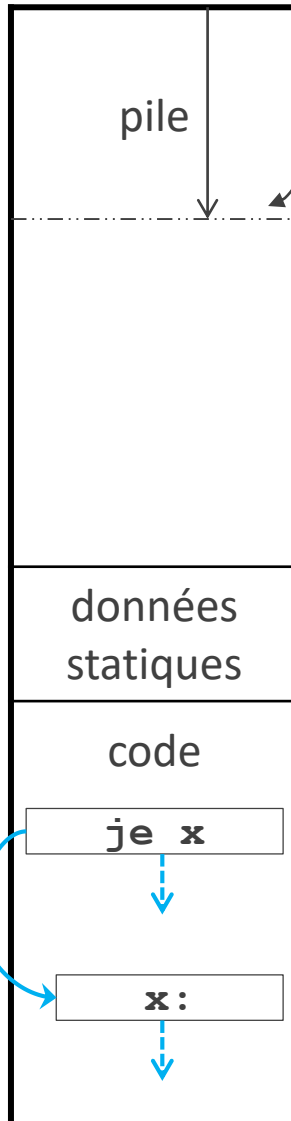
rsp

adresses

hautes

basses

saut



Révision : branchements

`jmp x`

inconditionnel

`je x`

conditionnel

`jne x`

`jg x`

`jge x`

Il n'existe aucune instruction de retour vers une
instruction de branchement

```
pop rbx
pop r12
pop r13
pop r14
call show_registers
mov r14, r13
mov r13, r12
mov r12, rbx
call show_registers
mov rax, 60
```

```
show_registers:
(...)
call printf
pop rbp
ret
```

Révision : appel et retour de fonction

Appel de fonction

Le moteur d'exécution

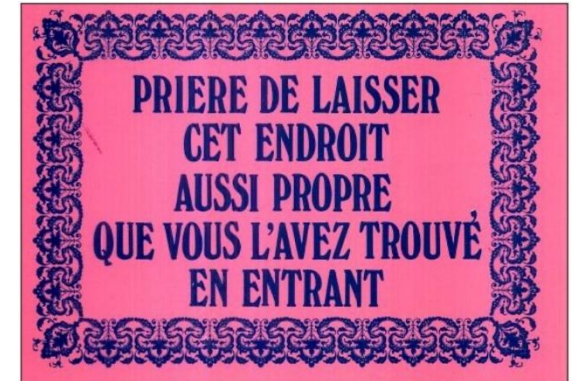
- empile l'adresse de retour
- saute à l'adresse de la fonction appelée

Retour d'une fonction

Le moteur d'exécution

- dépile l'adresse de retour
- saute à l'adresse de retour

Appel de fonction et retour



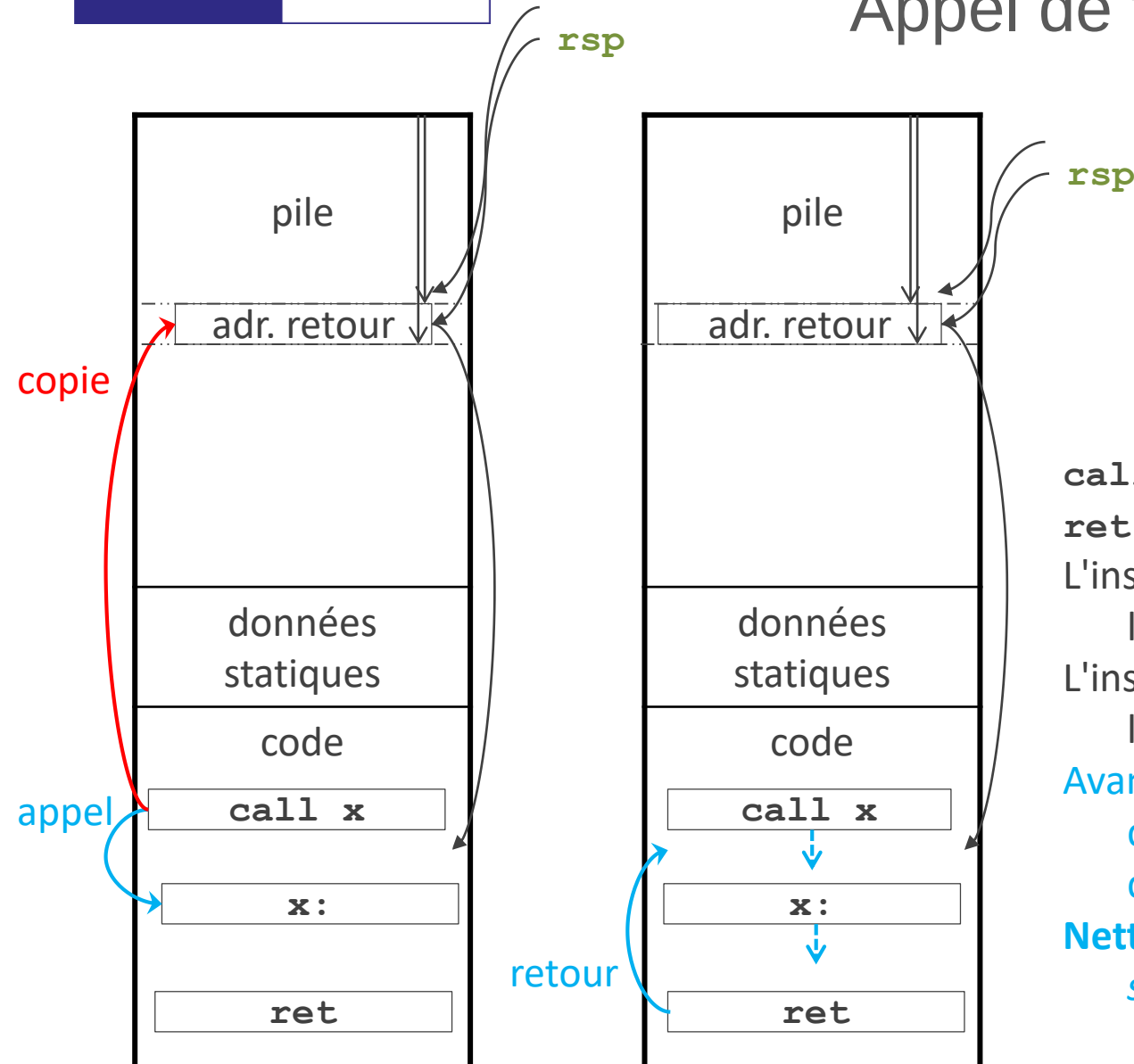
`call x` appeler une étiquette `x`
`ret` retour à l'appel

L'instruction `call` empile
 l'**adresse de retour**

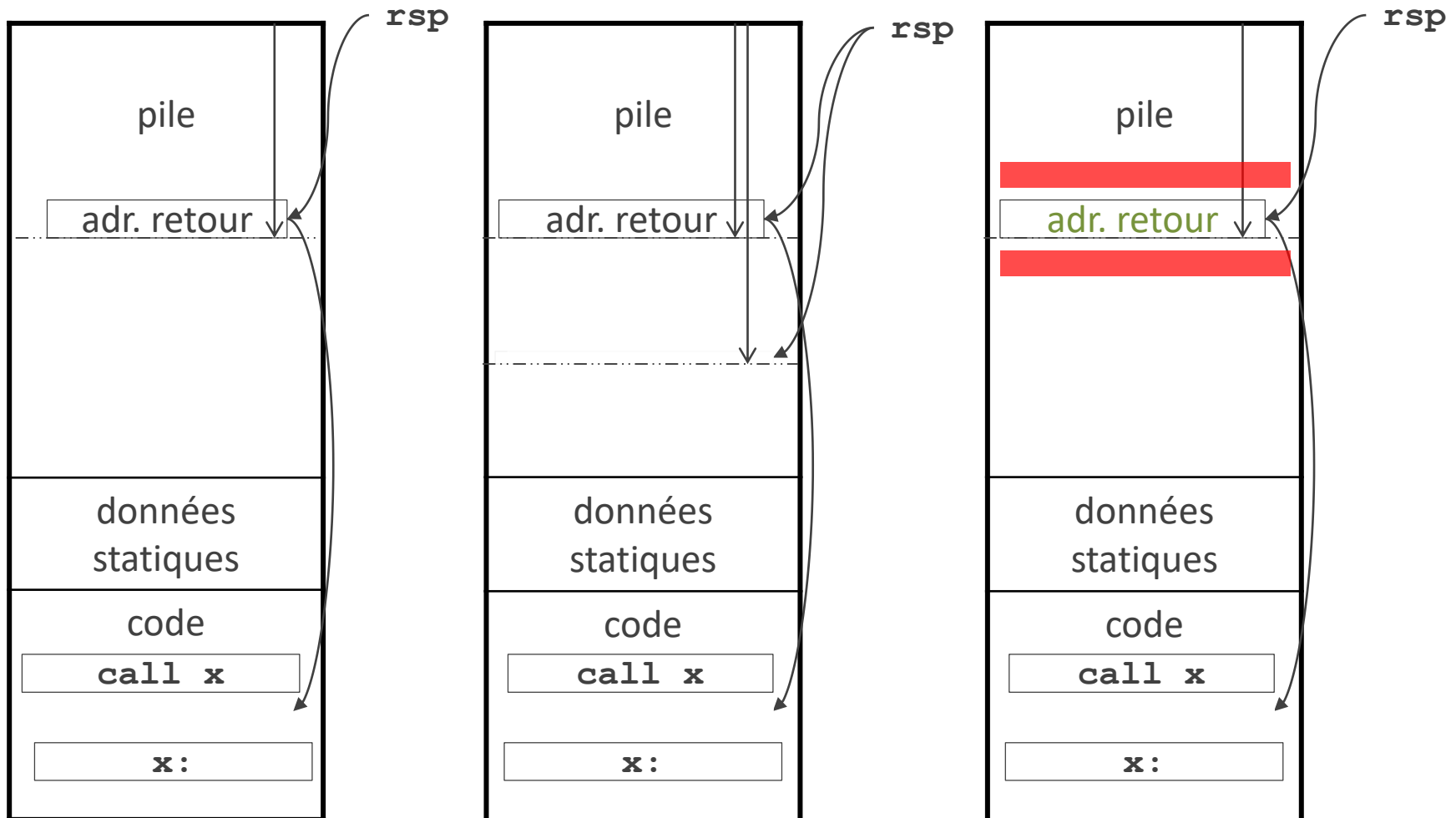
L'instruction `ret` dépile
 l'adresse de retour et l'utilise

Avant `ret`, il faut dépiler tout ce
 qu'on a empilé depuis le
 début de la fonction :

Nettoyer la pile (*clean up the
 stack*)



Appel de fonction et retour



Si on dépile trop ou pas assez, le système prend une mauvaise adresse de retour

Nettoyer (*clean up*) la pile avant le retour

```
my_function:
mov  rbx, 'element'
push rbx
mov  rbx, 'accident'
push rbx
mov  rbx, qword [rsp+4]
pop  rax
pop  rax
ret
```

Solution 1 (pour les cas simples)

Compter les `push` depuis le début de la fonction et faire le même nombre de `pop`

Difficultés

S'il y a des branchements conditionnels, le nombre de `push` par lesquels on est passé à l'exécution peut être imprévisible

Il faut aussi tenir compte des opérations autres que `push` qui affectent `rsp`

Nettoyer (*clean up*) la pile avant le retour

```
my_function:
push rbp
mov rbp, rsp
mov rbx, 'leraient'
push rbx
mov rbx, 'patrouil'
push rbx
mov rbx, qword [rsp+2]
mov rsp, rbp
pop rbp
ret
```

Solution 2 (infaillible)

Sauvegarder **rsp** dans **rbp** au début de la fonction

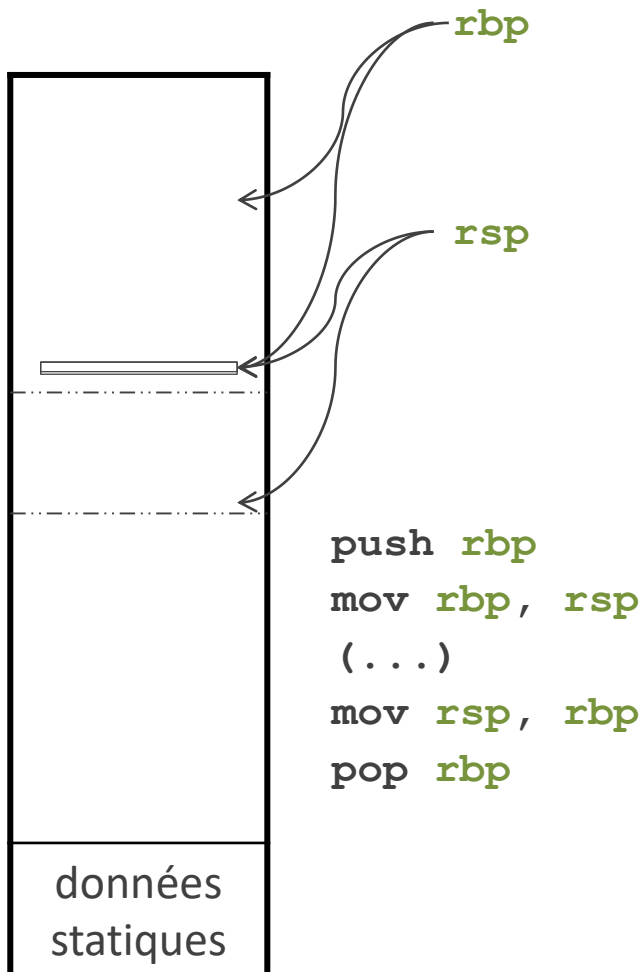
Restaurer **rsp** juste avant **ret**

Difficultés

rbp peut déjà contenir une sauvegarde de **rsp** pour une fonction dont l'activation est suspendue

Il faut le sauvegarder puis le restaurer

Le registre **rbp** (pointeur de base, *base pointer*)



Le registre **rbp** est fait pour indiquer où se trouve le sommet de pile vers le début de l'exécution de la fonction
C'est au développeur de le mettre à jour

Arguments et valeur de retour

Conventions d'appel

Mode d'emploi standard pour
les appels de fonctions dans
un certain environnement
(processeur, système, langage
à traduire)

Rien n'est prévu pour passer des arguments en
nasm

ni pour renvoyer une valeur de retour

On les met dans des registres le temps de l'appel
et du retour

Toutes les fonctions utilisent les mêmes registres

Comment éviter qu'une fonction écrase un
registre utilisé par une autre ?

Sans mode d'emploi, on s'expose à des bugs

Sommaire

Nettoyer la pile

Conventions d'appel AMD 64

Aligner la pile

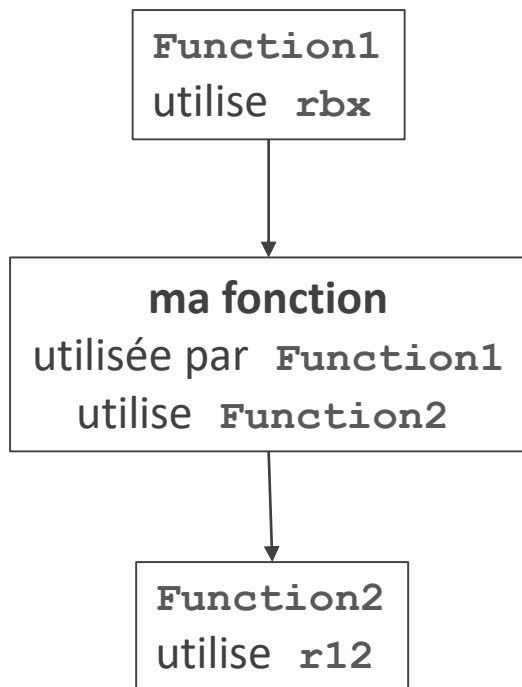
Instructions logiques bit à bit

Mélanger nasm et C

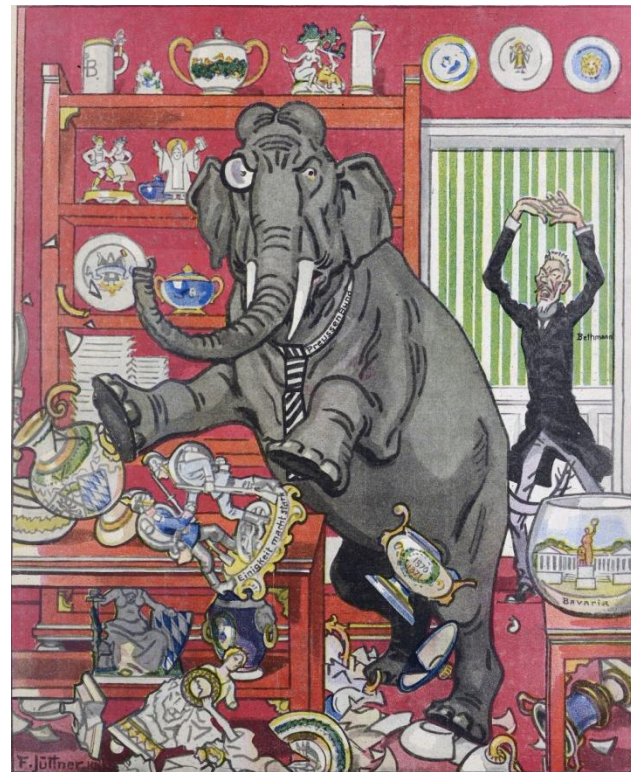
Conventions d'appel

Développer en assembleur est difficile

Pas de variables locales : tous les registres sont disponibles dans tout le code



Si ma fonction écrase **rbx**, elle peut provoquer un bug dans **Function1**
Si elle utilise **r12**, **Function2** peut provoquer un bug dans ma fonction



Franz Jüttner

Conventions d'appel

Développer en assembleur est difficile

- Pas de variables locales : tous les registres sont disponibles dans tout le code
- Les fonctions n'ont pas de paramètres ni de valeur de retour
- Certaines instructions imposent des contraintes sur l'état de la pile

Conventions d'appel

Discipline pour développer en assembleur

Dépend du processeur, du système d'exploitation
et du langage

Conventions d'appel

Conventions d'appel AMD 64

Valables quand 3 conditions sont réunies :

- processeurs X86-64
- Linux
- C ou C++

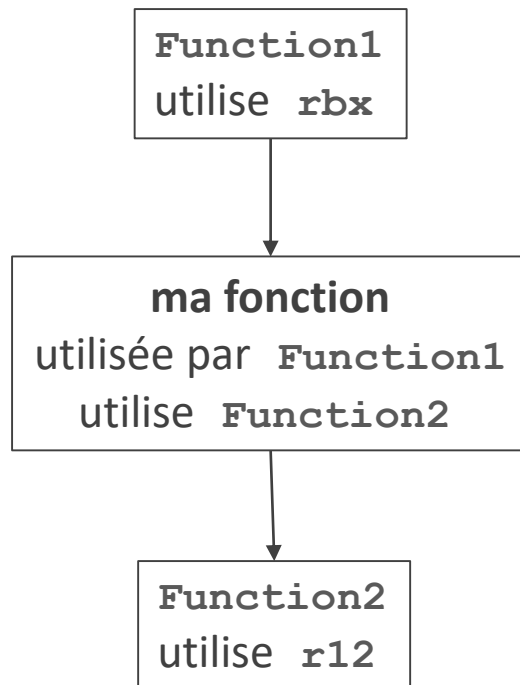
D'autres conventions sont valables dans d'autres contextes

C'est de la responsabilité des développeurs de les respecter

L'assembleur ne les vérifie pas

- **cdecl** avec processeurs IA-32
- **vectorcall** sous Windows
- **register** pour Delphi avec processeurs IA-32

Comment éviter d'écraser des registres



Pas de variables locales en assembleur

En appliquant les conventions AMD64, on n'a pas besoin de tenir compte à la fois des fonctions appelantes et des fonctions appelées

Registres non volatils

Tenir compte des fonctions appelantes

Registres volatils

Tenir compte des fonctions appelées

Comment éviter d'écraser des registres

Registres non volatils

On écrit `my_function`

On veut éviter que l'utilisation de `r12` provoque un bug dans la fonction appelante

Sauvegarder `r12` au début de la fonction appelée et le restaurer avant le retour

```
mov r12, qword 1968
mov r13, 0
mov r14, 0
call my_function
add rax, r12
mov rbx, rax
call show_registers
mov rax, 60
mov rdi, 0
syscall
```

```
my_function:
push r12
mov rbx, 'ellement'
push rbx
mov rbx, 'accident'
push rbx
mov rax, qword [rsp+4]
pop r12
pop r12
pop r12
ret
```


Comment éviter d'écraser des registres

Registres volatils

On veut appeler `my_function`

On veut éviter que l'exécution de `my_function` provoque un bug

Sauvegarder `r11` avant l'appel de fonction et le restaurer après

```
mov r11, qword 1968
mov r13, 0
mov r14, 0
push r11
call my_function
pop r11
add rax, r11
mov rbx, rax
call show_registers
mov rax, 60
mov rdi, 0
syscall
```

```
my_function:
mov rbx, 'ellement'
push rbx
mov rbx, 'accident'
push rbx
mov rax, qword [rsp+4]
pop r11
pop r11
ret
```

Conventions d'appel AMD 64

Registres non volatils (*callee-save*)

Les sauvegarder au début de la fonction appelée
et les restaurer avant le retour

Registres volatils (*caller-save*)

Les sauvegarder avant l'appel de fonction et les
restaurer après



By Ed Yourdon [CC BY-SA 2.0 (<http://creativecommons.org/licenses/by-sa/2.0>)], via Wikimedia Commons

my_function:

push rbp

mov rbp, rsp

(...)

mov rsp, rbp

pop rbp

ret

Registres non volatils (*callee-save registers*, *preserved registers*)

Le fait d'appeler une fonction ne les écrasera pas
Les fonctions qui les écrasent doivent restaurer au
retour la même valeur qu'à l'appel

rbx, rbp, rsp, r12, r13, r14, r15

Idéal dans du code

- qui ne peut pas être appelé (pas d'instruction **ret**)
- même s'il appelle des fonctions

Limites

Seulement quelques registres non volatils



By PiccoloNamek (English wikipedia) [GFDL (<http://www.gnu.org/copyleft/fdl.html>) or CC-BY-SA-3.0 (<http://creativecommons.org/licenses/by-sa/3.0/>)], via Wikimedia Commons

```
mov    qword [my_data], 7
mov    rax, rdi
add    rax, qword [my_data]
push   rax
push   rdi
call   show_stack
pop    rdi
pop    rax
mov    qword [my_data], rax
mov    rax, rdi
```

Registres volatils (*caller-save registers*, *scratch registers*)

Le fait d'appeler une fonction peut les écraser
Si on veut appeler une fonction alors qu'on les utilise, on doit les sauvegarder avant l'appel et les restaurer après

Tous les registres sauf **rbx**, **rbp**, **rsp** et **r12-15**

rax, **rcx**, **rdx**, **rdi**, **rsi**, **r8**, **r9**, **r10**, **r11**

Idéal dans une fonction qui n'appelle aucune fonction (fonction-feuille)

Arguments et valeur de retour

Rien n'est prévu pour indiquer

- combien d'arguments
- s'il y a une valeur de retour

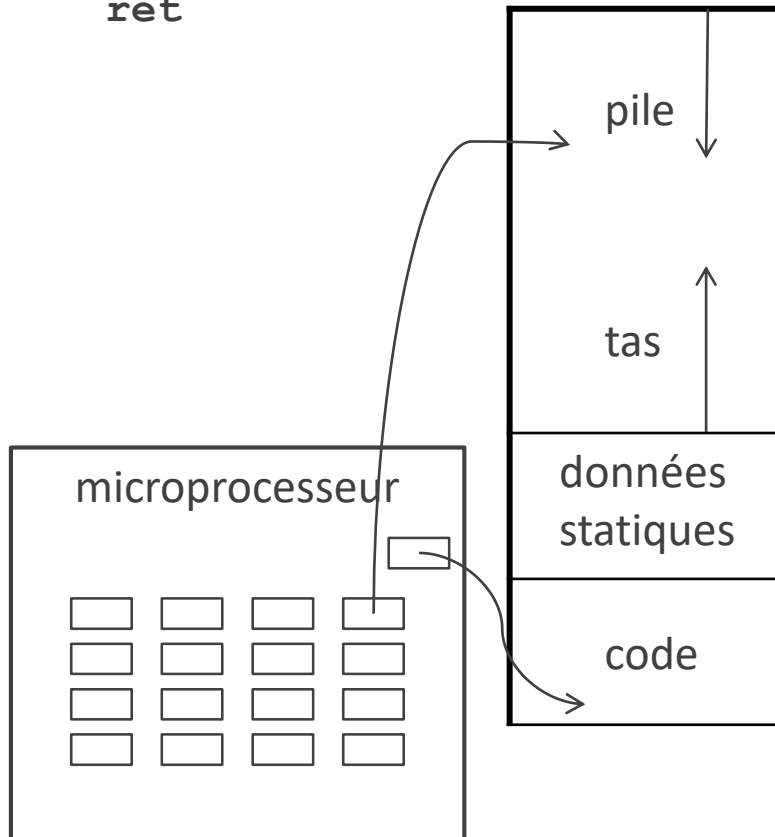
Rien n'est prévu pour passer des arguments en
nasm

ni pour renvoyer une valeur de retour

Les conventions d'appel donnent un mode
d'emploi au développeur et à l'utilisateur de la
fonction :

- où mettre les arguments à l'appel de la fonction
- où on trouve la valeur de retour après le retour
de la fonction

```
call my_function
(...)
my_function:
(...)
ret
```



Passer des paramètres et une valeur de retour

Syntaxe de l'assembleur

Instructions `call` et `ret`

Il faut sauvegarder quelque part les paramètres et la valeur de retour

Conventions d'appel AMD 64

On les sauvegarde à des emplacements précis dans les registres et la pile

On va les chercher là après

Conventions d'appel AMD 64

Les 6 premiers paramètres entiers et la valeur de retour entière

```
show_registers:
```

```
sub rsp, 8
```

```
mov r8, r14
```

```
mov rcx, r13
```

```
mov rdx, r12
```

```
mov rsi, rbx
```

```
mov rdi, format registers
```

```
mov rax, 0
```

```
call printf
```

```
add rsp, 8
```

```
ret
```

```
printf("...",x,y,z,t);
```

Les 6 premiers paramètres entiers

Dans les registres **rdi**, **rsi**, **rdx**, **rcx**, **r8** et **r9**, dans cet ordre

(Ici on met **rax** à 0 parce que **printf** est une fonction à nombre variable d'arguments)

La valeur de retour entière

Dans le registre **rax**

```
my_function:
```

```
mov rbx, 'element'
```

```
push rbx
```

```
mov rbx, 'accident'
```

```
push rbx
```

```
mov rax, qword [rsp+4]
```

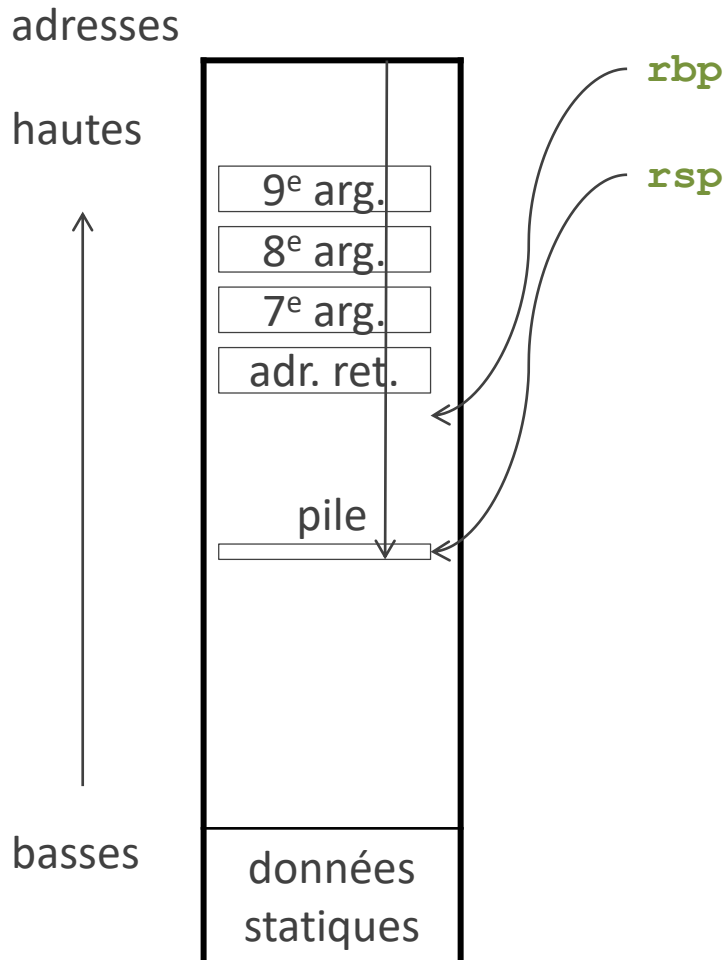
```
pop r11
```

```
pop r11
```

```
ret
```

Conventions d'appel AMD 64

Autres paramètres entiers



S'il y a plus de 6 paramètres entiers

Sur 8 octets chacun

Empilés avant l'appel

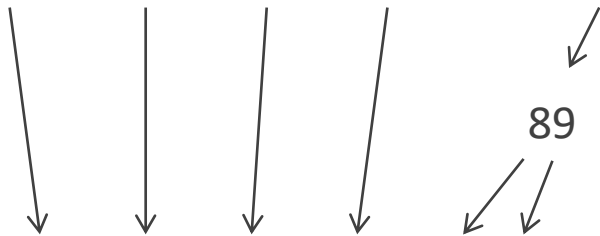
En commençant par le dernier (pour que l'adresse relative par rapport à **rbp** ne dépende pas du nombre d'arguments)

Les 6 premiers paramètres entiers

Le DJ signe à la radio récemment pour des hits neufs

Dix sirènes radieuses récoltent des huitres neuves

Dis-moi si la radio raconte la révolution



`rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`

**Pour se souvenir de la liste des registres des 6
premiers paramètres entiers**

Laquelle des trois phrases marche le mieux ?

Votez sur <https://xoyondo.com/ap/j7nELTvPGQW7jn2>

Sommaire

[Nettoyer la pile](#)

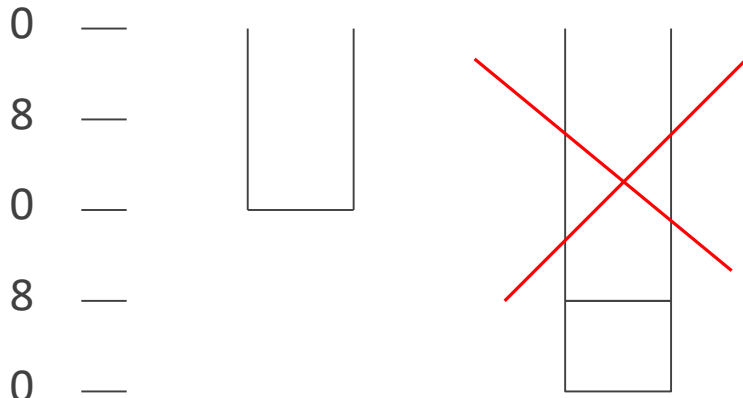
Conventions d'appel AMD 64

Aligner la pile

Instructions logiques bit à bit

Mélanger nasm et C

Aligner la pile



Les conventions d'appel AMD 64 imposent que la taille de la pile soit un multiple de 16 octets juste avant un appel

Sinon on risque une erreur de segmentation à l'exécution

Cela facilite l'emploi de certaines instructions qui présupposent que l'adresse d'un opérande est un multiple de 16

Aligner la pile sur un multiple de 16

On doit pouvoir rétablir le sommet après le retour

La pile est alignée au début de l'exécution du programme

Aligner la pile

Solution 1 (cas simples)

```
show_registers:
sub rsp, 8
mov r8, r14
mov rcx, r13
mov rdx, r12
mov rsi, rbx
mov rdi, format_registers
mov rax, 0
call printf
add rsp, 8
ret
```

← Si la pile était alignée juste avant l'appel à `show_registers`, elle est à nouveau alignée ici

Compter les opérations qui affectent la pile

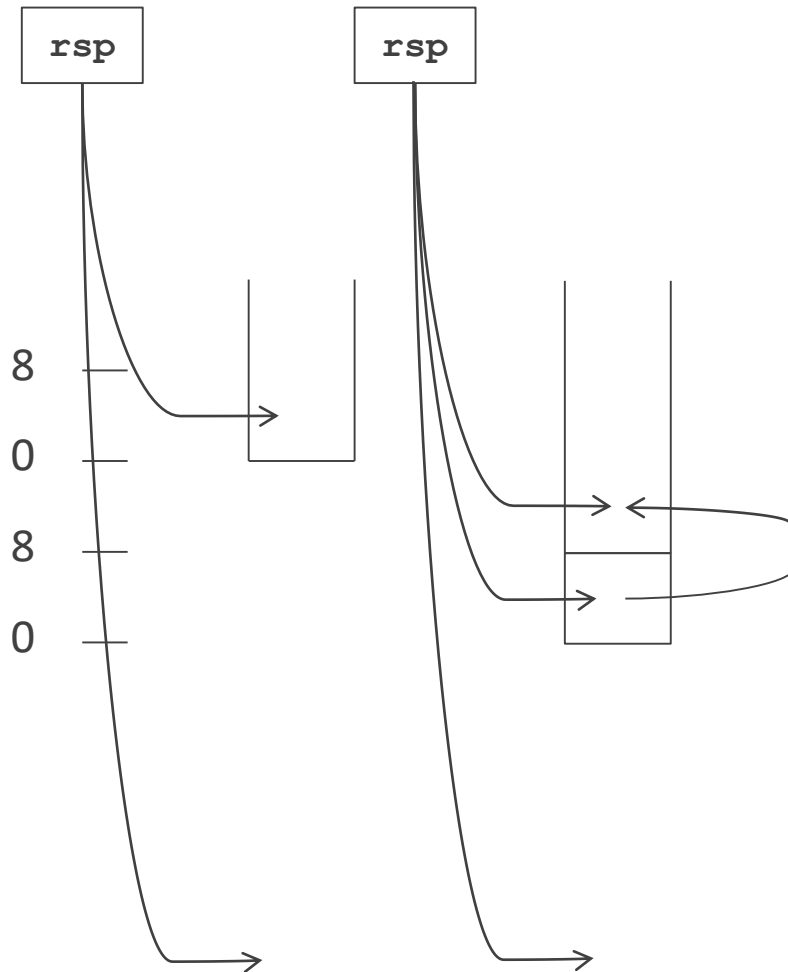
- L'appel empile automatiquement 8 octets
- Les `push` et les `pop`
- Les autres opérations sur `rsp`

Aligner la pile avant un appel

Restaurer la pile après l'appel

Aligner la pile

Solution 2 (infaillible)

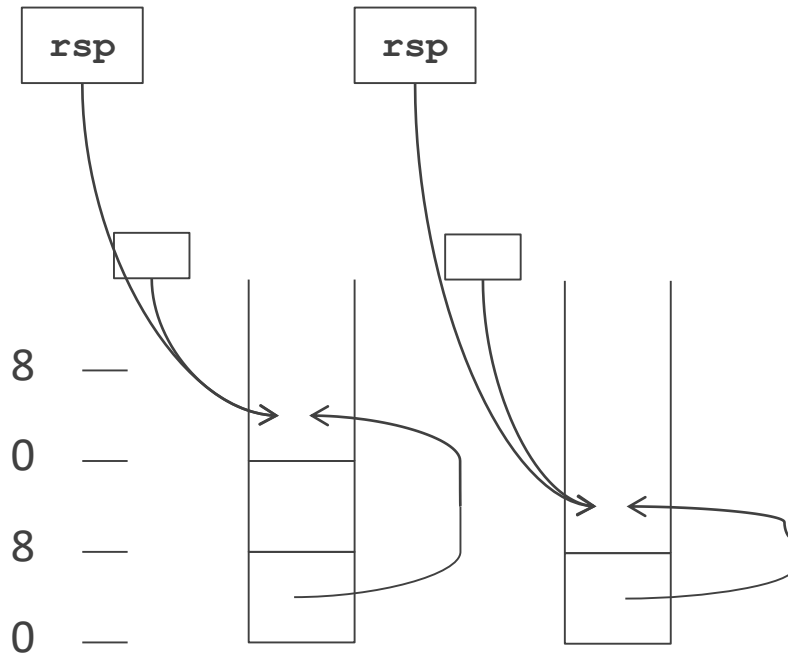


On ne peut pas toujours prévoir la taille de la pile
Si on sauvegarde **rsp** dans un registre et que les appels s'empilent, il va falloir un registre à chaque appel

- Sauvegarder **rsp** dans la pile
- Aligner la pile
- Appeler la fonction
- Restaurer **rsp**

Comment faire sans sauts ?

Aligner la pile Solution 2 (infaillible)



- Sauvegarder provisoirement **rsp** dans un autre registre
- Réserver dans la pile de quoi sauvegarder **rsp**
- Aligner la pile
- Sauvegarder l'ancien **rsp** dans la pile
- Appeler la fonction
- Restaurer **rsp**

Aligner la pile

Solution 2 (infaillible)

```
mov r11, rsp
sub rsp, 8
;align stack here
mov qword [rsp], r11
call my_function2
pop rsp
```

- Sauvegarder provisoirement `rsp` dans un autre registre
- Réserver dans la pile de quoi sauvegarder `rsp`
- Aligner la pile
- Sauvegarder l'ancien `rsp` dans la pile
- Appeler la fonction
- Restaurer `rsp`

Aligner la pile

Solution 2 (infaillible)

```
mov r11, rsp
sub rsp, 8
mov rdx, 0
mov rax, rsp
mov rbx, 16
idiv rbx ; remainder in rdx
sub rsp, rdx
mov qword [rsp], r11
call my_function2
pop rsp
```

Comment placer dans `rsp` le plus grand multiple de 16 inférieur ou égal à `rsp` ?

On peut utiliser la division entière de `rsp` par 16

Sommaire

[Nettoyer la pile](#)

Conventions d'appel AMD 64

Aligner la pile

Instructions logiques bit à bit

Mélanger nasm et C

Instructions logiques bit à bit

and bitwise and

or bitwise or

xor bitwise exclusive or

xor rax, rax ; est plus rapide que **mov rax, 0**

not bitwise negation

Exercice

Écrire du code pour vérifier si
les bits de rang 5 et 7 de **rflags**
(en comptant les rangs de 0 à 7)
sont tous les deux à 0

Application : aligner la pile

```
mov r11, rsp
sub rsp, 8
and rsp, -16
mov qword [rsp], r11
call my_function2
pop rsp
```

Comment placer dans `rsp` le plus grand multiple
de 16 inférieur ou égal à `rsp` ?

Mettre à 0 les 4 bits de poids faible de `rsp`

`0xffff_ffff_ffff_fff0`

-16 étendu à `0xffff_ffff_ffff_fff0`

(si on écrit directement

`0xffff_ffff_ffff_fff0` on a un

avertissement parce que la constante est
tronquée à 4 octets)

Sommaire

[Nettoyer la pile](#)

Conventions d'appel AMD 64

Aligner la pile

Instructions logiques bit à bit

Mélanger nasm et C

Appeler du C depuis nasm

```
extern printf
section .data
    display_format db "rbx:%ld r12:%ld r13:%ld r14:%ld", 10, 0
section .text
show_registers:
sub rsp, 8 ; align stack
mov r8, r14
mov rcx, r13
mov rdx, r12
mov rsi, rbx
mov rdi, display_format
mov rax, 0
call printf WRT ..plt
add rsp, 8 ; restore stack
ret
```

- Déclarer le nom de la fonction en **extern**
- Utiliser l'instruction **call** avec le nom de la fonction et éventuellement **WRT ..plt**
- Respecter les conventions d'appel :
 - placer les arguments dans les registres avant l'appel
 - aligner la pile
 - récupérer la valeur de retour après l'appel

Appeler du nasm depuis C

```
#include <stdio.h>
#include <inttypes.h>
int64_t maxof3(int64_t, int64_t, int64_t);
int main() {
    printf("%ld\n", maxof3(2, 3, 1));
    printf("%ld\n", maxof3(2, -6, 5));
    return 0;
}
```

```
global maxof3
section .text
maxof3:
    mov rax, rdi
    cmp rax, rsi
    jge yless
        mov rax, rsi
yless:
    cmp rax, rdx
    jge zless
        mov rax, rdx
zless:
    ret
```

Déclarer le prototype de la fonction
On est sûr que `int64_t` fait 8 octets
Utiliser le nom de la fonction comme étiquette
Respecter les conventions d'appel :

- lire les arguments dans les registres
- placer la valeur de retour dans `rax`

Pour l'édition de liens avec `gcc`, ne pas utiliser
l'option `-nostartfiles`

Merci

CONTACT

ERIC LAPORTE

00 +33 (0)1 60 95 75 52

ERIC.LAPORTE@UNIV-EIFFEL.FR